PROGRAMMING LANGUAGES

Java

A comprehensive, interview-ready tour of modern Java: the JVM, OOP, generics, collections, streams, concurrency, and build tooling.

java · jvm · oop · collections · generics · streams · concurrency

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What Java Is & Why It Matters
2. Setup & Getting Started
3. Types, Variables & the Memory Model
4. Object-Oriented Programming
5. Generics
6. The Collections Framework
7. The Streams API & Lambdas
8. Exceptions & Error Handling
9. Concurrency Basics
10. Build Tools: Maven & Gradle
11. Worked Mini-Example: Word Frequency Counter
12. Cheat-Sheet & Python Comparison
13. Best Practices
14. Common Pitfalls
15. Interview Questions & Answers
16. Further Learning

What Java Is & Why It Matters

Java is a statically typed, object-oriented, garbage-collected language that compiles to a portable intermediate form (bytecode) and runs on the Java Virtual Machine. Its promise since 1995 has been "write once, run anywhere": the same compiled artifact runs unmodified on Linux, Windows, macOS, servers, and phones.

Java dominates large-scale backend systems, Android apps, big-data tooling (Hadoop, Spark, Kafka, Elasticsearch), and enterprise services (Spring). Its appeal is a combination of a strong static type

system that catches errors at compile time, a mature just-in-time (JIT) compiling VM that reaches near-native speed on hot code paths, automatic memory management, and one of the deepest ecosystems and labour markets in software. As of 2026 the current long-term-support (LTS) release is **Java 21** (with Java 25 as the newest LTS), and modern Java is far more concise than the verbose reputation it earned in the 2000s.

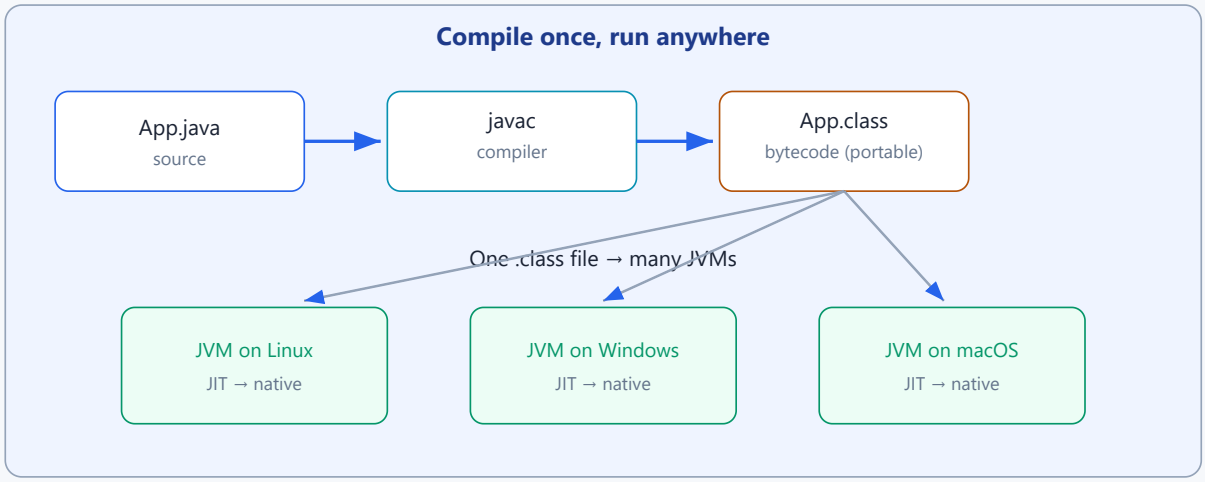


Fig 1 — The Java toolchain: javac emits portable bytecode once; each platform's JVM JIT-compiles it to native code at runtime.

> **Python contrast**
> Python is dynamically typed and interpreted (CPython), with duck typing and an implicit runtime. Java forces you to declare types up front, catches whole classes of bugs at compile time, and typically runs 5-30x faster on CPU-bound code thanks to the JIT — at the cost of more ceremony and longer feedback loops.

Setup & Getting Started

Install a JDK (Java Development Kit, which includes `javac`, the JVM, and tooling). A build tool like Maven or Gradle is standard for anything beyond a toy. The easiest path on any OS is SDKMAN!.

BASH

```
# Install SDKMAN! then a JDK (Temurin is a popular free OpenJDK build)
curl -s "https://get.sdkman.io" | bash
sdk install java 21.0.5-tem      # install a specific LTS
sdk install maven
sdk install gradle

java -version                    # verify the runtime
javac -version                   # verify the compiler
```

BASH

```
# Classic compile-and-run (no build tool)
javac Hello.java                 # produces Hello.class
java Hello                       # runs the class (no .class extension)

# Java 11+ can run a single source file directly, no javac step:
java Hello.java

# Java 21+ "instance main" lets you skip the ceremony entirely for scripts
java --enable-preview Script.java
```

JAVA

```
// Hello.java  — the traditional entry point
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
        for (String arg : args) {
            System.out.println("arg: " + arg);
        }
    }
}
```

Tip

Use an LTS release (21 or 25) for anything you ship. Non-LTS releases (22, 23, 24...) are fine for experimenting with new features but receive only six months of updates.

Types, Variables & the Memory Model

Java has two kinds of types: **primitives** (stored by value, on the stack or inline) and **reference types** (objects, where the variable holds a reference to a heap-allocated object). Understanding this split explains most Java surprises.

PRIMITIVES (8 OF THEM)

`byte` (8-bit), `short` (16), `int` (32), `long` (64), `float` (32-bit IEEE), `double` (64), `char` (16-bit UTF-16 unit), `boolean`. No methods, never null, fixed size.

REFERENCE TYPES

Everything else: `String`, arrays, your classes, `Integer`, `List`. The variable is a handle; the object lives on the garbage-collected heap. Can be `null`.

JAVA

```java
int count = 42;                 // primitive, lives inline
Integer boxed = count;          // autoboxing: primitive → wrapper object
int back = boxed;               // unboxing: wrapper → primitive

long big = 9_000_000_000L;      // underscores for readability; L suffix for long
double pi = 3.14159;
char c = 'J';
boolean ok = true;

var name = "Alice";             // Java 10+ local type inference (still static!)
final int MAX = 100;            // final = cannot be reassigned

String s = null;                // references can be null
// int n = null;                // COMPILE ERROR: primitives cannot be null
```

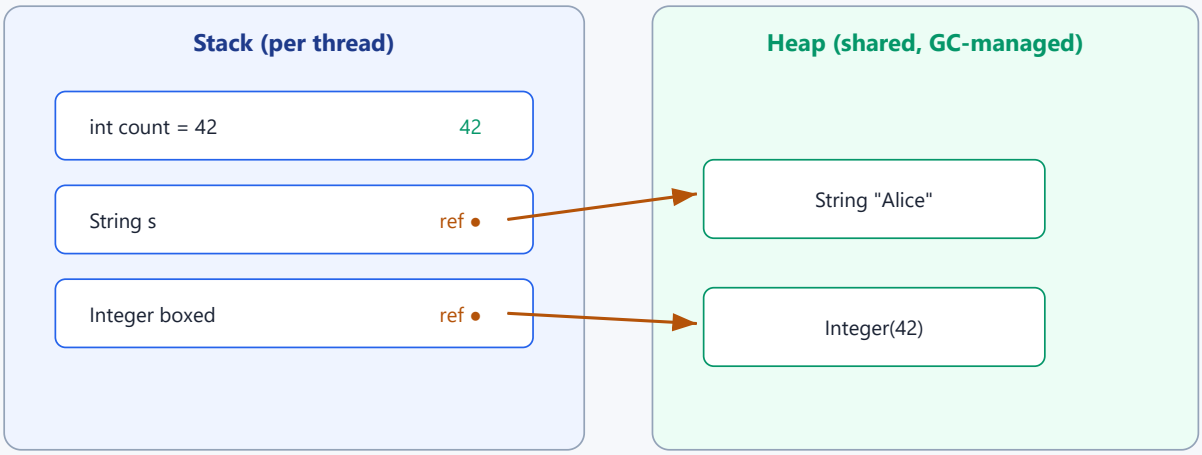


Fig 2 — Primitives store their value directly in the stack frame; reference variables store a pointer to an object on the heap.

⚠ **Pitfall: == vs equals**

For reference types, `==` compares *references* (identity), while `.equals()` compares *value*. `"a" + x == "ax"` is often false; always use `.equals()` for String/object content comparison. Primitives use `==` for value.

Object-Oriented Programming

Java is class-based OOP to its core. The four pillars — **encapsulation**, **inheritance**, **polymorphism**, and **abstraction** — map directly to language features.

JAVA

```java
// Encapsulation: private fields + public accessors
public class Account {
    private double balance;              // hidden state
    private final String owner;          // immutable after construction

    public Account(String owner, double opening) {
        this.owner = owner;
        this.balance = opening;
    }

    public void deposit(double amount) {
        if (amount ≤ 0) throw new IllegalArgumentException("must be positive");
        balance += amount;
    }

    public double getBalance() { return balance; }   // controlled read access
}
```

JAVA

```java
// Inheritance + polymorphism
public abstract class Shape {
    abstract double area();                          // subclasses must implement
    public String describe() { return "area=" + area(); }
}

public class Circle extends Shape {
    private final double r;
    public Circle(double r) { this.r = r; }
    @Override double area() { return Math.PI * r * r; }
}

public class Square extends Shape {
    private final double side;
    public Square(double side) { this.side = side; }
    @Override double area() { return side * side; }
}

// Polymorphism: one reference type, many runtime behaviours
Shape s = new Circle(2.0);
System.out.println(s.describe());        // dispatches to Circle.area() at runtime
```

INTERFACE

A contract of method signatures. A class can implement many. Can hold `default` and `static` methods since Java 8.

ABSTRACT CLASS

Partial implementation + state. A class extends exactly one. Use when subclasses share code.

RECORD

Java 16+ immutable data carrier. One line replaces constructor, getters, `equals`, `hashCode`, `toString`.

JAVA

```java
// interface with a default method
public interface Greeter {
    String name();
    default String greet() { return "Hello, " + name(); }  // shared default
}

// record: immutable data class in a single line (Java 16+)
public record Point(int x, int y) {
    public double distanceTo(Point o) {
        return Math.hypot(x - o.x, y - o.y);
    }
}
Point p = new Point(3, 4);
System.out.println(p.x() + "," + p.y());   // auto-generated accessors
System.out.println(p);                     // Point[x=3, y=4]  (auto toString)

// sealed types + pattern matching (Java 21) - exhaustive, no default needed
sealed interface Expr permits Num, Add {}
record Num(int v) implements Expr {}
record Add(Expr l, Expr r) implements Expr {}

static int eval(Expr e) {
    return switch (e) {
        case Num n -> n.v();
        case Add a -> eval(a.l()) + eval(a.r());
    };
}
```

> **Python contrast**
>
> Python uses duck typing ("*if it quacks...*") and multiple inheritance with MRO. Java requires explicit `implements` / `extends`, forbids multiple class inheritance (but allows many interfaces), and resolves method calls through a strongly typed vtable.

Generics

Generics give you type-safe containers and algorithms without casting. A `List<String>` can only hold Strings; the compiler enforces it and inserts casts for you.

JAVA

```java
// Generic class
public class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

Box<String> b = new Box<>();     // diamond operator infers <String>
b.set("hi");
String v = b.get();              // no cast needed

// Generic method with a bounded type parameter
public static <T extends Comparable<T>> T max(List<T> items) {
    T best = items.get(0);
    for (T it : items) if (it.compareTo(best) > 0) best = it;
    return best;
}

// Wildcards: ? extends (producer, read) vs ? super (consumer, write) — "PECS"
void printAll(List<? extends Number> nums) {        // can READ as Number
    for (Number n : nums) System.out.println(n);
}
void addInts(List<? super Integer> sink) {          // can WRITE Integers
    sink.add(42);
}
```

⚠ **Type erasure**

Generics are a compile-time construct; at runtime `List<String>` and `List<Integer>` are both just `List`. You cannot do `new T[]` or `x instanceof List<String>`. This is called *erasure* and is a favourite interview topic.

The Collections Framework

The `java.util` collections are the daily bread of Java. Know the interface hierarchy and the performance of each implementation cold.

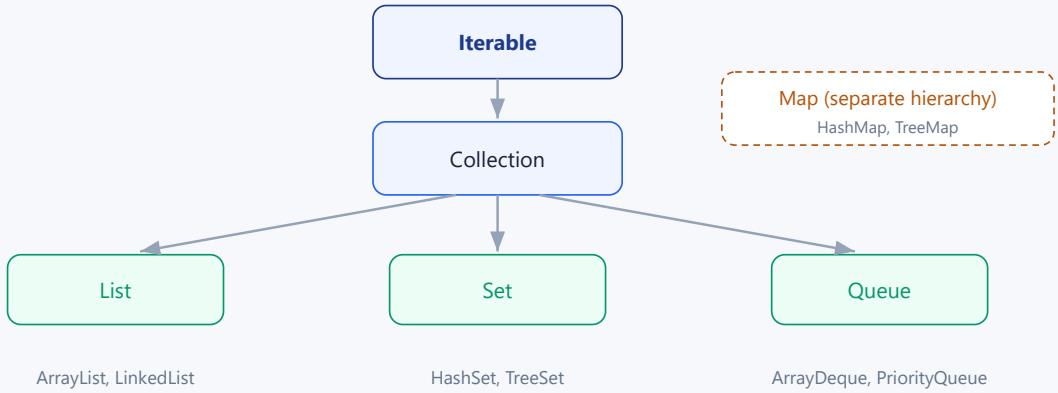


Fig 3 — The collections interface hierarchy. Note Map is NOT a Collection — it is a separate root.

JAVA

```java
import java.util.*;

List<String> list = new ArrayList<>();     // ordered, indexed, allows dups
list.add("a"); list.add("b");
String first = list.get(0);

Set<Integer> set = new HashSet<>();        // unique elements, no order
set.add(1); set.add(1);                    // second add is a no-op
boolean has = set.contains(1);

Map<String,Integer> map = new HashMap<>(); // key → value
map.put("apples", 3);
map.merge("apples", 1, Integer::sum);      // idiomatic counter increment → 4
int n = map.getOrDefault("pears", 0);

Deque<Integer> stack = new ArrayDeque<>(); // use as stack OR queue
stack.push(10); stack.push(20);
int top = stack.pop();                     // 20

// Immutable factory methods (Java 9+)
List<String> fixed = List.of("x", "y", "z");   // throws on modification
```

Type	Backing	get/contains	add/insert	Ordered?	Use when
ArrayList	resizable array	O(1) index	O(1) amortized end	insertion	default list, random access
LinkedList	doubly linked	O(n)	O(1) at ends	insertion	frequent head/tail ops
HashMap / HashSet	hash table	O(1) avg	O(1) avg	no	default map/set
TreeMap / TreeSet	red-black tree	O(log n)	O(log n)	sorted	need sorted order / ranges
LinkedHashMap	hash + list	O(1) avg	O(1) avg	insertion/access	LRU caches, stable iteration
ArrayDeque	circular array	—	O(1) ends	insertion	stack or queue (beats Stack/LinkedList)

> **Tip**
> Any object used as a `HashMap` key or `HashSet` element MUST override both `equals()` and `hashCode()` consistently. `record` types do this for you automatically — one more reason to prefer them for keys.

The Streams API & Lambdas

Java 8 introduced lambdas and the Streams API, bringing a functional, declarative style for processing sequences. Streams express *what* you want (filter, map, reduce) rather than *how* to loop.

JAVA

```java
import java.util.*;
import java.util.stream.*;

List<String> names = List.of("Alice", "Bob", "Carol", "Dave");

// Filter, transform, collect — reads top-to-bottom like a pipeline
List<String> result = names.stream()
    .filter(n -> n.length() > 3)           // keep long names
    .map(String::toUpperCase)              // method reference
    .sorted()
    .collect(Collectors.toList());         // → [ALICE, CAROL, DAVE]

// Numeric reduction
int totalLen = names.stream().mapToInt(String::length).sum();

// Grouping — one of the most useful collectors
Map<Integer, List<String>> byLen = names.stream()
    .collect(Collectors.groupingBy(String::length));

// Reduce to a single value
Optional<String> longest = names.stream()
    .max(Comparator.comparingInt(String::length));

// Lambdas implement functional interfaces (single-abstract-method)
Runnable r = () -> System.out.println("run!");
Comparator<String> byLenCmp = (a, b) -> a.length() - b.length();
```



> ⚠ **Pitfall**
>
> Streams are lazy and single-use. Intermediate ops (`filter` , `map`) do nothing until a terminal op (`collect` , `forEach` , `count`) runs. Reusing a consumed stream throws `IllegalStateException` . Avoid side effects inside stream pipelines.

Exceptions & Error Handling

Java splits exceptions into **checked** (must be declared or caught — e.g. `IOException`) and **unchecked** (`RuntimeException` and subclasses — e.g. `NullPointerException` , `IllegalArgumentException`). `Error` (e.g. `OutOfMemoryError`) you generally never catch.

JAVA

```java
// try-with-resources: auto-closes anything implementing AutoCloseable
import java.io.*;

static String readFirstLine(String path) throws IOException {   // checked, declared
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        return br.readLine();
    }   // br.close() called automatically, even on exception
}

// try / catch / finally with multi-catch
try {
    int x = Integer.parseInt(userInput);
    process(x);
} catch (NumberFormatException | IllegalStateException e) {   // multi-catch
    System.err.println("Bad input: " + e.getMessage());
} finally {
    cleanup();   // always runs
}

// Custom exceptions — extend RuntimeException for unchecked
public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String msg) { super(msg); }
}
```

Danger

Never write an empty `catch (Exception e) {}`. Swallowing exceptions hides bugs. At minimum log it; usually rethrow or wrap. Also avoid catching `Throwable` — it traps `Error`s you cannot recover from.

Concurrency Basics

The JVM has true OS threads and a mature concurrency toolkit. Modern Java (21+) adds **virtual threads**, making it cheap to run millions of blocking tasks.

JAVA

```java
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

// Prefer an ExecutorService over raw new Thread()
try (ExecutorService pool = Executors.newFixedThreadPool(4)) {
    Future<Integer> f = pool.submit(() -> expensiveCompute());
    Integer result = f.get();      // blocks until done
}   // pool.close() shuts down and awaits (Java 19+ AutoCloseable)

// Virtual threads (Java 21): millions of cheap threads for blocking I/O
try (var vpool = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        vpool.submit(() -> { doBlockingIo(); return null; });
    }
}

// Shared mutable state needs synchronization
private final AtomicInteger counter = new AtomicInteger();   // lock-free
counter.incrementAndGet();

private int balance = 0;
public synchronized void deposit(int n) { balance += n; }    // intrinsic lock

// CompletableFuture: compose async work
CompletableFuture.supplyAsync(() -> fetch())
    .thenApply(data -> transform(data))
    .thenAccept(System.out::println);
```

CONCURRENT COLLECTIONS

Use `ConcurrentHashMap`, `CopyOnWriteArrayList`, and `BlockingQueue` instead of manually locking plain collections.

VISIBILITY: VOLATILE

`volatile` guarantees reads/writes of a field are visible across threads (no caching), but does NOT make compound actions atomic.

> **Python contrast**
> CPython has a Global Interpreter Lock (GIL) that serializes bytecode, so threads do not run Python in parallel. The JVM has no GIL — Java threads achieve genuine multi-core parallelism, which is why Java is favoured for CPU-bound concurrent workloads.

Build Tools: Maven & Gradle

Real projects use a build tool to manage dependencies, compile, test, and package. Maven uses declarative XML; Gradle uses a Groovy/Kotlin DSL and is faster via incremental builds and caching.

XML

```xml
<!-- Maven: pom.xml -->
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>myapp</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.11.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

BASH

```
# Maven lifecycle
mvn compile          # compile sources
mvn test             # run tests
mvn package          # build a JAR into target/
mvn clean install    # wipe, build, install to local repo (~/.m2)
```

GROOVY

```
// Gradle: build.gradle (Groovy DSL)
plugins { id 'java' }
java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }
repositories { mavenCentral() }
dependencies {
    testImplementation 'org.junit.jupiter:junit-jupiter:5.11.0'
}
// Run with:  ./gradlew build   ./gradlew test   ./gradlew run
```

Worked Mini-Example: Word Frequency Counter

A small end-to-end program tying together collections, streams, and modern syntax: count the most frequent words in a block of text.

JAVA

```java
import java.util.*;
import java.util.stream.*;

public class WordCount {
    public static void main(String[] args) {
        String text = """
            the quick brown fox the lazy dog
            the fox jumps the dog sleeps
            """;                                    // text block (Java 15+)

        Map<String, Long> freq = Arrays.stream(text.toLowerCase().split("\\s+"))
            .filter(w -> !w.isBlank())
            .collect(Collectors.groupingBy(w -> w, Collectors.counting()));

        // Top 3 by count, descending
        freq.entrySet().stream()
            .sorted(Map.Entry.<String,Long>comparingByValue().reversed())
            .limit(3)
            .forEach(e -> System.out.printf("%-6s %d%n", e.getKey(), e.getValue()));
    }
}
// Output:
// the    4
// fox    2
// dog    2
```

Cheat-Sheet & Python Comparison

Task	Java	Notes
Print	`System.out.println(x)`	or `printf` with format strings
Format string	`"x=%d".formatted(n)`	or `String.format`
Make list	`List.of(1,2,3)`	immutable; use `new ArrayList<>()` for mutable
Iterate	`for (var x : list)`	enhanced for-loop
Map entry loop	`for (var e : map.entrySet())`	`e.getKey()` / `e.getValue()`
Null-safe	`Optional<T>`	avoid returning null from APIs
Immutable data	`record Point(int x,int y){}`	Java 16+
Switch value	`var r = switch(x){ … }`	arrow form, returns a value

Concept	Java	Python
Typing	static, compile-checked	dynamic, duck typing
Execution	bytecode + JIT (fast)	interpreted (CPython)
Memory	GC, primitives vs objects	GC, everything an object
Parallelism	real threads, no GIL	GIL serializes threads
Lists	`List<T>` (typed)	`list` (any)
Dicts	`Map<K,V>`	`dict`
Lambda	`x -> x+1`	`lambda x: x+1`
None/null	`null` (refs only)	`None`

Best Practices

✓ **Do**

Program to interfaces (`List<String> x = new ArrayList<>()`), prefer immutability (`final` , records), favour composition over inheritance, and return `Optional` instead of null for "maybe absent" values.

✦ **Tip**

Use `var` for obvious local types to cut noise, but keep explicit types in public APIs. Let the IDE and compiler guide you — Java's static types make refactoring safe and automated.

✓ **Do**

Write tests with JUnit 5. Keep methods short and single-purpose. Use `equals` / `hashCode` / `toString` (or records) on any value type.

Common Pitfalls

⚠ **== on objects**

Comparing Strings or boxed numbers with `==` compares identity, not value. The `Integer` cache (-128..127) makes this bug intermittent and maddening. Always use `.equals()` .

⚠ **Mutating during iteration**

Modifying a collection while a for-each loop iterates throws `ConcurrentModificationException` . Use an `Iterator.remove()` or `removeIf()` instead.

✕ **NullPointerException**

The most common runtime failure. Validate inputs, prefer `Optional` , use `Objects.requireNonNull` , and enable helpful NPE messages (default since Java 15).

Interview Questions & Answers

What is the difference between the JDK, JRE, and JVM?

The JVM is the runtime engine that executes bytecode. The JRE (now folded into the JDK) is the JVM plus standard libraries needed to run apps. The JDK is the full development kit: JRE plus compiler (`javac`), debugger, and tools. You develop with the JDK; you run on the JVM.

Explain `==` versus `.equals()`.

`==` compares primitives by value and objects by reference (identity). `.equals()` compares logical equality as defined by the class. For custom value types you must override `equals` (and `hashCode` alongside it) or use a record.

Why must you override `hashCode` whenever you override `equals`?

Hash-based collections (`HashMap` , `HashSet`) locate objects by their hash code first, then confirm with `equals` . If two equal objects return different hash codes, they land in different buckets and the collection breaks (duplicates, failed lookups). The contract: equal objects must have equal hash codes.

What is type erasure?

Generics exist only at compile time. The compiler checks types and inserts casts, then erases the type parameters, so `List<String>` becomes raw `List` at runtime. Consequences: you cannot do `new T[]` , cannot use `instanceof List<String>` , and overloads differing only by generic type are illegal.

Checked vs unchecked exceptions — when to use each?

Checked exceptions (extend `Exception`) must be declared or caught and model recoverable conditions (file not found). Unchecked (`RuntimeException`) model programming errors (bad argument, null) and need no declaration. Modern practice leans toward unchecked exceptions to avoid boilerplate; use checked only when the caller can realistically recover.

What does `final` mean in its three positions?

On a variable: cannot be reassigned (the reference is fixed, though a mutable object it points to can still change). On a method: cannot be overridden. On a class: cannot be subclassed (e.g. `String`).

How does the garbage collector decide what to free?

An object is eligible for GC when it is no longer reachable from any GC root (stack variables, static fields, active threads). Modern collectors (G1, ZGC, Shenandoah) are generational and mostly

concurrent, minimizing pause times. You never free memory manually; you can only drop references.

ArrayList vs LinkedList — which and why?

Prefer `ArrayList` almost always: O(1) random access and cache-friendly contiguous memory. `LinkedList` only wins for frequent insertion/removal at both ends — and even there `ArrayDeque` is usually faster. In practice LinkedList is rarely the right answer.

What are virtual threads and why do they matter?

Introduced in Java 21, virtual threads are lightweight threads scheduled by the JVM onto a small pool of OS carrier threads. They make blocking I/O cheap — you can run millions concurrently — letting you write simple synchronous code that scales like async, without callback hell or reactive frameworks.

What is the difference between an interface and an abstract class?

A class implements many interfaces but extends only one abstract class. Interfaces define contracts (plus default/static methods) and hold no instance state; abstract classes can hold state and constructors and share implementation. Choose an interface for a capability, an abstract class for a partial base implementation.

Explain the PECS rule for generics wildcards.

"Producer Extends, Consumer Super." Use `? extends T` when a structure produces values you read as T; use `? super T` when it consumes values you write as T. This maximizes flexibility of generic method signatures while staying type-safe.

Is Java pass-by-value or pass-by-reference?

Always pass-by-value. For objects, the value passed is a copy of the reference, so the method can mutate the pointed-to object but cannot reassign the caller's variable. There is no pass-by-reference in Java.

Further Learning

- **Effective Java** (Joshua Bloch) — the canonical best-practices book; read it twice.
- **Modern features** — practice records, sealed types, pattern matching in `switch`, and virtual threads (Java 21+).
- **Spring Boot** — the dominant framework for Java web services and microservices.
- **Concurrency in Practice** (Goetz) — deep dive into the memory model and thread-safety.
- **JVM internals** — learn GC tuning, JIT, and tools like `jcmd`, `jstack`, and JDK Flight Recorder.

- **Build & test** — get fluent with Maven/Gradle, JUnit 5, and Mockito.